Soft Actor-Critic (SAC) no Hopper-v5 — relatório experimental

Vinícius Nogueira Tormin e **Bruno Calura** · Aprendizado por Reforço, UFG · orçamento de 100 mil passos de ambiente

Fork com `algorithms/sac.py` e os configs usados (`configs/sac_hopper.yml`, `configs/sac_hopper_tuned.yml`; os overrides de cada braço estão em `tools/run_sweep_sac.py`): github.com/viniciustormin/t-zero-learning · W&B: sem conta; runs gravados offline no projeto `sac-assignment` (`wandb/offline-run-*`, `exp_name` = braço)

Protocolo. `tests/test_sac.py` 16/16 antes de qualquer treino. Sanidade no Pendulum (`sac_pendulum`, 20k): `eval/mean_return` = **−129,9** (> −200). Corrigimos uma linha **fora** do algoritmo: o `train.py` imprimia `args.algo.learning_rate`, que o `SACConfig` não tem (`policy_lr/q_lr`), e todo treino SAC quebrava. Cada **braço** = um config + overrides (`tools/run_sweep_sac.py`); 3 seeds por config na Parte 3, 2 nas Partes 4–5. Sem conta no W&B: `WANDB_MODE=offline`, e as figuras vêm do espelho local de cada `wandb.log` (`tools/train_csv.py` → `sweep_results_sac/<braço>__seed<N>/metrics.jsonl`), e **todo número** sai de `tools/summarize_sac.py` (trajetórias: `--trace`). Diário completo: `report/tuning_log_sac.md`; previsões, escritas antes de cada experimento: `report/predictions_sac.md`. Até 8 runs em paralelo (núcleos P, 1 thread cada); **tempo e SPS só vêm dos runs isolados** `bench_*`. **ret100** = `charts/episodic_return_mean_last100` no fim; **eval** = `eval/mean_return` (10 episódios, ação média). Ambos como média (mín–máx) entre seeds.

Parte 3 — Eficiência amostral no Hopper (100k passos, 3 seeds)

O que o `base` mostra. (i) `losses/alpha` cai igual nos 3 seeds: 0,90 no 1º log (5,1k) → 0,41 (6,1k) → 0,18 (8,1k) → 0,10 (10,1k), e depois oscila entre 0,06 e 0,16 (mediana na 2ª metade: 0,0999 / 0,0911 / 0,0739). **(ii)** A entropia passa de −2,0 em 10k para o alvo −3 a partir de ≈20k e fica nele (−3,00 / −3,00 / −3,01 nos últimos 20%). **(iii)** `qf1_values` sobe de 45–49 (10k) para ≈200, acompanhando `soft_return_ep_start`, exceto em **base/1 entre 70k e 80k**: ret100 705 → 68, episódios de 259 → 93 passos, `q_minus_return` ≈ 195 (Q ≈ 200 contra G ≈ 60), depois de picos de `qf1_loss` de 52 e 57 (os outros seeds ficam ≤ 25). **(iv)** Em 100k, base/2 e base/3 **ainda sobem** +126 a +167 de ret100 a cada 10k (episódios de 321 e 401 de 1000 passos). **(v)** O eval fica muito acima do ret100 em base/1 e base/3 (2869,4 contra 542,0; 2965,1 contra 1194,5), porque a janela de 100 episódios cobre ≈30k passos, e é muito ruidoso: o **mesmo** modelo de base/1 deu 2869,4 e, no run isolado, 2699,6. **(vi)** 4,8 min por run com 4 em paralelo.

mudança (antigo → novo)	motivo observado (run)	eval	ret100	veredito
config inicial (`sac_hopper`)	—	2274,2 (988,2–2965,1)	950,8 (542,0–1194,5)	referência
`sac.updates_per_step` 1 → 4	base/2 e base/3 ainda ganham +126 a +167 de ret100 a cada 10k perto de 100k: falta passo de gradiente, não dado	1776,4 (1006,9–2447,4)	1288,3 (1109,5–1434,7)	indistinguível (mais próximo)
`sac.updates_per_step` 1 → 2	a mesma leitura, a metade do custo	1034,1 (15,6–2793,3)	946,8 (437,3–1353,5)	indistinguível
`sac.learning_starts` 5000 → 1000	nos 3 seeds o `alpha` só chega a 0,10 em 10,1k, ≈5k passos de gradiente depois do início	1004,7 (973,3–1035,6)	1127,6 (965,7–1426,9)	indistinguível
`sac.q_lr` 1e-3 → 3e-4	base/1 colapsa em 70–80k com `q_minus_return` ≈ 195 e picos de `qf1_loss` de 52–57: crítico mais lento	834,3 (347,6–1405,8)	736,1 (335,3–1076,8)	indistinguível (tendência de piora)

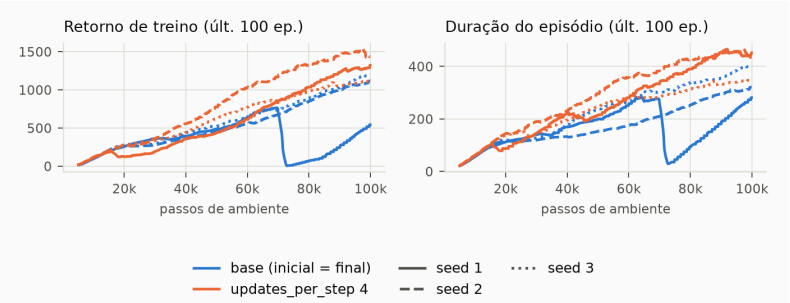


Fig. 1 — Config inicial (= final) × o candidato mais próximo, 3 seeds cada (linha = seed, sem suavização). O degrau azul sólido é o colapso de base/1 em 70k.

Veredito: nenhum candidato passou do ruído, e o config final é o inicial (`configs/sac_hopper_tuned.yml`, corpo idêntico ao `sac_hopper.yml`). O `updates_per_step 4` tem o maior ret100, mas a diferença de médias (+337,5) é menor que a amplitude do base (652,5), e base/2 e base/3 caem dentro do intervalo dele; no eval, a média dele é **menor**. Os demais se sobrepõem por inteiro. Os colapsos tardios dominam a variância: r1_ups2/2 cai de 1163 para 437 entre 90k e 100k, e a política final cai em ≈15 passos (eval 15,6 nos 10 episódios), com Q1 parado em ≈219. É o mesmo padrão de base/1. O `learning_starts 1000` faz o que prometia no `alpha` (< 0,1 em 6k em vez de 10k), mas isso não aparece no retorno. Sem vencedores, não houve rodada 2.

Parte 4 — O que cada peça faz (config final, seeds 1–2)

4.1 — Sem ajuste automático: `sac.autotune=false`, `sac.alpha=0.09`. Escolha: a mediana do `losses/alpha` dos 3 seeds do base na 2ª metade (0,0999 / 0,0911 / 0,0739) é 0,09, o nível em que o autotune estaciona. Assim a ablação tira o **ajuste**, não o nível. **Previsão:** entropia abaixo do alvo no começo (o base tem α de 0,9 a 0,1 até 10k) e **derivando para −4 a −6**, porque a escala de Q sobe de 45–49 para ≈200 e nada compensa; retorno indistinguível. **Resultado:** acertei o começo (entropia de −2,66 / −2,10 em 10k, contra −2,06 / −1,87 do base) e o retorno (eval 2120,5 (1073,4–3167,6) contra 1928,8 (988,2–2869,4); ret100 567,4 (435,2–699,7) contra 829,0 (542,0–1115,9), sobrepostos; abl_alpha/1 colapsa entre 80k e 90k, de 1141,9 para 351,5, como base/1). **Errei a deriva:** a entropia fica 0,3–0,5 abaixo do alvo entre 20k e 50k (−3,53 / −3,45) e volta a −3,30 / −3,00 depois de 50k. **Linhas:** com `autotune=false`, `self.alpha` é fixado uma vez em `sac.py:340` e a atualização de `sac.py:496` nunca roda, então o mecanismo que segura a entropia, `compute_alpha_loss` (`sac.py:205`, que sobe α quando $-\log\pi$ fica abaixo de −3), some. A deriva não veio porque a perda do ator (`sac.py:194`) depende de $\partial Q/\partial a$, não do nível de Q: o Q sobe pelo valor do estado, que é comum a todas as ações.

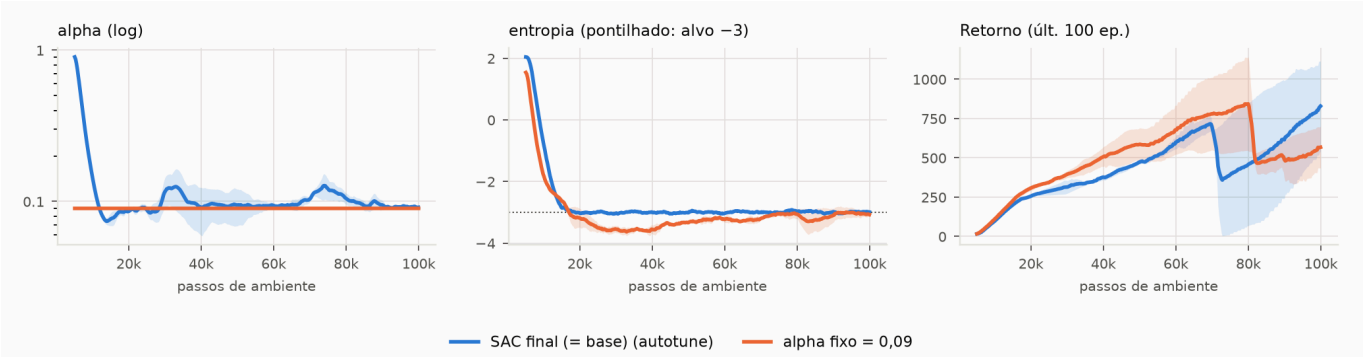


Fig. 2 — Ablação 1 (média e envoltória dos seeds 1–2). No base, α parte de 1 (log $\alpha = 0$) e chega ao regime em ≈10k; a entropia começa em +2 (ações quase uniformes) e encosta em −3 por volta de 20k.

4.2 — Um crítico só: `sac.clipped_double_q=false`. Previsão: superestimação crescente (`q_minus_return` > 0 e subindo, `qf1_values` acima do base) e retorno pior ou mais instável, com o efeito em Q mais claro que no retorno. **Resultado:** a superestimação veio, e grande: Q1 = 543,7 / 517,2 contra 197,4 / 210,4; `q_minus_return` médio de 494,3 / 471,1 nos últimos 20% (base: 181,4 / −26,0, o primeiro inflado pelo colapso), `qf1_loss` ≈ 10× (242,1 / 184,6). **Errei o retorno:** com 2 seeds ele não piora (eval 2446,9 (2084,2–2809,6), ret100 1057,3 (901,7–1212,8)). Também não previ que o `alpha` **triplicasse** (mediana ≥ 50k: 0,320 / 0,295 contra 0,100 / 0,091). **Linhas:** `pick_q2` (`sac.py:366`) passa a devolver Q1 no lugar de Q2, e por isso o `torch.min` do alvo da Parte 2 (`sac.py:178`, chamado em `sac.py:454`) e o da perda do ator (`sac.py:194`, chamada em `sac.py:482`) viram `min(Q1, Q1)` = Q1. O erro positivo de Q1 é **escolhido** pelo ator e **propagado** pelo alvo. O ator, que persegue o máximo de um Q1 inflado, fica mais concentrado, e `compute_alpha_loss` (`sac.py:205`) sobe α até a entropia voltar a −3,00. Como boa parte do viés é um deslocamento comum a todas as ações, a ordem entre ações quase não muda, e em 100k passos o retorno não sente.

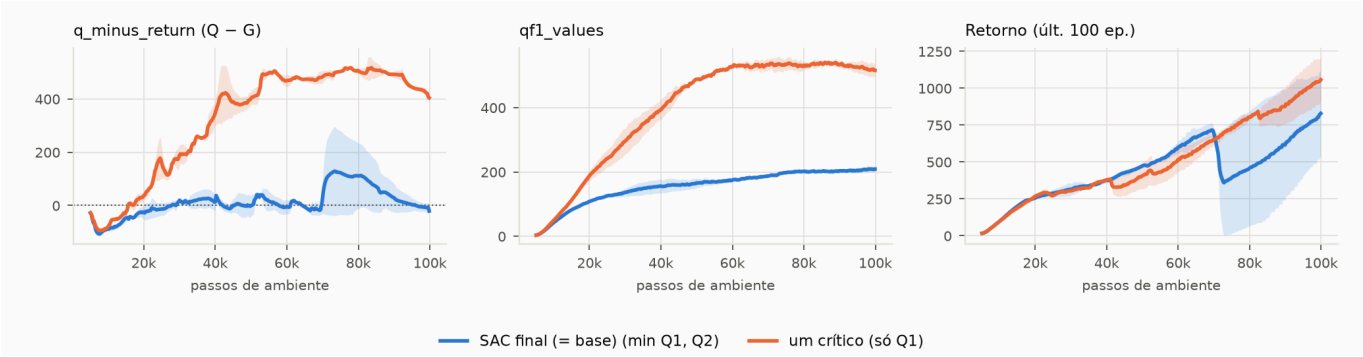


Fig. 3 — Ablação 2 (seeds 1–2). `q_minus_return` = Q do 1º par (s, a) do episódio menos o retorno **soft** descontado que o episódio rendeu. A faixa azul em 70–90k é o colapso de base/1.

Parte 5 — SAC × PPO no Hopper (100k passos)

PPO = `ppo_pendulum` com `env_id=Hopper-v5`, `total_timesteps=100000`, `gamma=0.99`, sem ajuste. Ele termina em 98.304 passos (24 iterações × 4096). SAC = config final (= base). Mostramos também o `updates_per_step 4` da Parte 3, para ver o efeito em tempo.

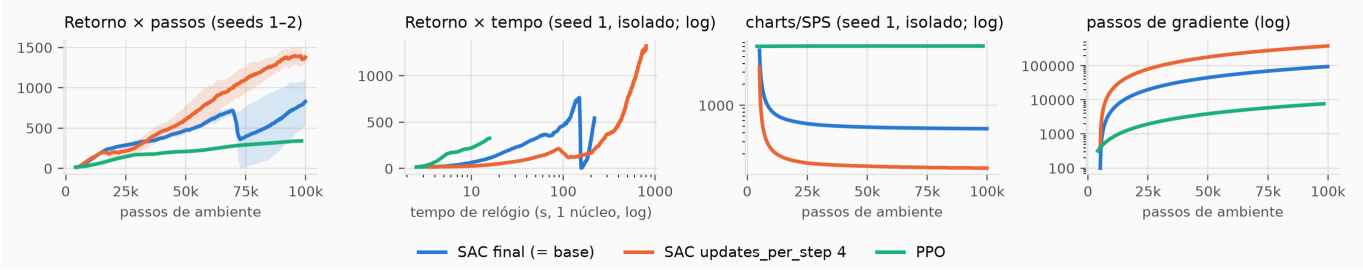


Fig. 4 — SAC × PPO. Painéis 1 e 4: seeds 1–2 (média e envoltória). Painéis 2 e 3: seed 1 de cada, rodado sozinho na máquina (`bench_ppo`, `bench_sac_final`, `bench_sac_ups4`). Eixo do tempo em log: o PPO inteiro cabe em 16 s.

	passos até ret100 300	ret100 final	eval	tempo total	SPS	passos de grad.	ret100 em 20 s / 60 s / 120 s	usos por transição
SAC final	23,5k / 32,5k	542,0 / 1115,9	2869,4 / 988,2	220 s	460	94.900	151,8 / 355,5 / 559,4	242,9
SAC ups 4	36,7k / 27,2k	1320,9 / 1434,7	2447,4 / 1006,9	800 s	125	379.600	50,8 / 142,0 / 128,0	971,8
PPO	90,1k / 69,6k	325,8 / 356,1	338,4 / 368,0	**16 s**	**7105**	7.680	**325,8** (fim)	10,0

Seeds 1 / 2. Tempo, SPS e ret100 em 20/60/120 s: run isolado do seed 1 (treino determinístico: `bench_sac_final` repete o ret100 de base/1, e `bench_ppo` o do ppo_hopper/1).

Eficiência amostral. Com os mesmos passos, **o SAC chega muito mais longe**: ret100 300 em 23,5k–32,5k passos contra 69,6k–90,1k do PPO, e no fim 542,0–1194,5 (3 seeds) contra 325,8–356,1; eval 988,2–2965,1 contra 338,4–368,0.

Tempo de relógio. O PPO roda a 7105 SPS e o SAC a 460 (15,4×). O SAC dá 94.900 passos de gradiente (um por passo de ambiente, `sac.py:424`) contra 7.680 do PPO (12,4×), cada um com uma amostra do buffer, três redes e duas perdas. **Com o mesmo tempo do PPO**

(16 s), o PPO vai mais longe: em 20 s o SAC está em 151,8. O SAC passa de 300 em 44 s e já está em 355,5 em 60 s, acima de onde o PPO parou. **Com ≥ 1 min, o SAC vai mais longe.** O `updates_per_step 4` multiplica por 4 os passos de gradiente (`sac.py:424–425`) e deixa o SAC 3,7× mais lento (125 SPS): ele fica **atrás** do base em todo tempo intermediário (142,0 contra 355,5 em 60 s; 128,0 contra 559,4 em 120 s) e só o passa depois dos 220 s do base. Ou seja, empurra a resposta para o lado do PPO, e na Parte 3 nem pagou em retorno por passo acima do ruído.

Mecanismo. **(1) Replay off-policy** explica a eficiência por passo **e** o custo por passo. Cada transição entra, em média, em (100k − 5k) × 1 × 256 / 100k = **242,9** passos de gradiente do crítico (94.900 × 256 / 100.000, `sac.py:447`); o ator reusa os mesmos batches (2 passos a cada 2 do crítico, `policy_frequency 2` compensado). No PPO são **10,0** = `update_epochs`: cada época embaralha o batch de 4096 e passa por todos os 32 minibatches de 128 (`ppo_continuous_action.py:286`). Conferindo com o `num_updates`: 7.680 × 128 / 98.304 = 10,0. São ≈24× mais usos por transição, e o PPO descarta tudo depois de cada iteração, porque a sua perda é on-policy. **(2) Ator reparametrizado** (`rsample`, `sac.py:147`): o gradiente do ator passa **por dentro** de Q ($\partial Q/\partial a$), um sinal de baixa variância que só existe porque há um crítico de ação treinado naquele replay; o PPO usa a razão de probabilidades com vantagens de um rollout. **(3) Dois críticos** explicam a estabilidade de Q, não o retorno em 100k: sem o mínimo, Q superestima em ≈480 (Parte 4.2), mas o retorno não caiu. **(4) Entropia:** o alvo −3 mantém a política explorando sem precisar de ajuste (o α vai sozinho de 0,9 para ≈0,1), enquanto o PPO termina com entropia de 3,68 / 3,64 (desvio-padrão ainda alto). Com α fixo no nível certo (4.1), nada muda em 100k; o que o autotune compra é não precisar escolher esse nível.